

Examen réparti DLP

Carlos Agon – 2016

Conditions générales

Le barème est fixé à 20 ; la durée de l'épreuve est de 2 heures. Tous les documents sont autorisés et notamment ceux du cours. Tous les appareils électroniques communicants sont prohibés (et donc notamment les téléphones portables). Vos oreilles ne doivent pas être reliées à ces appareils.

Votre copie sera formée de fichiers textuels que vous laisserez aux endroits spécifiés dans votre espace de travail pour Eclipse. L'espace de travail pour Eclipse sera obligatoirement nommé `workspace` et devra être un sous-répertoire direct de votre répertoire personnel.

L'examen sera corrigé à la main, il est donc absolument inutile de s'acharner sur un problème de compilation ou sur des méthodes à contenu informatif faible. Il est beaucoup plus important de rendre aisé, voire plaisant, le travail du correcteur et de lui indiquer, par tout moyen à votre convenance, de manière claire, compréhensible et terminologiquement précise, comment vous surmontez cette épreuve. À ce sujet, vos fichiers n'auront que des lignes de moins de 80 caractères, n'utiliseront que le codage ASCII ou UTF-8 enfin, s'abstiendront de tout caractère de tabulation.

Le langage à étendre est ILP2. Le paquetage Java correspondant sera nommé `com.paracamplus.ilp2.partial_16`. Sera ramassé, à partir de votre *workspace* (situé sous ce nom directement dans votre HOME), le seul répertoire `ILP-UPMC/Java/src/com/paracamplus/ilp2/partial_16` et tout ce qu'il contient.

Exercice 1 – Les paramètres optionnels [20/20]

Dans cet exercice, nous voulons étendre ILP2 avec l'utilisation des *paramètres optionnels* pour l'appel et la définition de fonctions. Cette nouvelle caractéristique permet parfois de simplifier les appels de fonction, au détriment de la simplicité de leur définition. L'exemple suivant illustre l'utilisation des *paramètres optionnels* :

Commençons par définir une fonction de manière traditionnelle (en ILP2).

```
function toto (a,b,c,d) (print(a); print(b); print(c); print(d); newline());
```

L'appel `toto(1,2,3,4)` donnera comme résultat :

```
1234
```

Définissons maintenant une fonction équivalente, mais avec des *paramètres optionnels* :

```
function toto1 (a,b &keys c 3, d 4) (print(a); print(b); print(c); print(d); newline());
```

Le mot-clé `&keys` divise les paramètres de la fonction en deux groupes : les paramètres traditionnels et les *paramètres optionnels*. Un *paramètre optionnel* est composé d'un nom et d'une valeur par défaut (cette valeur peut uniquement être une constante). Dans notre exemple il y a deux *paramètres optionnels* :

- `c` avec valeur par défaut 3.
- `d` avec valeur par défaut 4.

Puisqu'ils ont une valeur par défaut, les *paramètres optionnels* peuvent être omis dans l'appel de fonction, par exemple, `toto1(1,2)` donnera comme résultat :

```
1234
```

Il est aussi possible de passer une expression comme argument d'un *paramètre optionnel*. Dans ce cas l'appel de fonction utilise le caractère spécial `:` suivi du nom du *paramètre optionnel* puis une expression qui définira sa valeur.

Le résultat de `toto1(1,2, :d 4+1)` est :

```
1235
```

L'ordre des *paramètres optionnels* n'a pas d'importance, pour `toto1(1,2, :d 5, :c 6)` on aura :

```
1265
```

Par contre, les *paramètres optionnels* doivent toujours être placés après les arguments normaux.

L'appel `toto1(1, :d 5, 3)` produit une erreur.

1. Question - Stratégie [3/20]

Dans un fichier texte appelé `strategie.txt`, écrire les étapes à suivre pour interpréter et compiler des programmes ILP incluant les *paramètres optionnels*. Donner la liste des interfaces et des classes qui doivent être ajoutées, préciser de quelles classes elles dérivent ou quelles interfaces elles implémentent. Indiquer les interfaces et les classes existantes qui doivent être modifiées et dans quel sens.

Livraison : un fichier `strategie.txt`.

2. Question - Grammaire [3/20]

Étendre la grammaire d'ILP2 en incluant les *paramètres optionnels*.

Livraison : un fichier `ILPMLgrammar_partial_16.g4`.

3. Question - Interprétation [5/20]

Écrire les classes nécessaires pour l'interprétation des programmes incluant les *paramètres optionnels*.

Livraison : les fichiers correspondant à ces classes seront placés dans le paquetage `partial_16.interpreter`.

4. Question - Compilation [6/20]

Écrire les nouveaux schémas de compilation pour la définition et l'appel des fonctions (inspirez vous des slides du cours 4). Pour l'appel de fonction, expliquer dans quelles conditions la gestion des arguments peut se faire dynamiquement et quand elle doit se faire statiquement. Proposer des idées pour l'implémentation de ces deux cas.

Livraison : un fichier texte `compilation.txt`. Concernant les schémas de compilation, il est inutile d'ajouter le code en Java qui génère du C. Vous pouvez adopter une convention personnelle pour représenter les flèches avec du texte ainsi que d'autres éléments non textuels du schéma.

5. Question - Test [3/20]

En utilisant JUnit 4, écrire une classe de test pour les *paramètres optionnels*. Vous serez évalué sur la quantité et la qualité des tests proposées.

Livraison : un fichier `PartialTest.java`.